get_next_line — Hard Trace Example

Prepared by Daniel Landi

Scenario:
- **File content:** `"AB\nC"`
- `BUFFER_SIZE`: 2
- **Action:** `get_next_line(fd)` is called repeatedly until exhaustion.

Call 1 — First Execution

Init: `static char *reserve` is initialized to `NULL` (first run). `read_to_reserve(fd, NULL)` is called.

Step 1: `read_to_reserve` (Accumulation)

- **Iteration 1:**
  - `bucket = malloc(3)` is allocated.
  - `read(fd, bucket, 2)` returns 2 bytes (`b = 2`).
  - `bucket` becomes `"AB\0"`.
  - `reserve = ft_strjoin(NULL, "AB")` creates a new string: `"AB"`.
- **Iteration 2:**
  - The loop checks `!ft_strchr("AB", '\n')`. It is true (no newline yet), so it continues.
  - `read(fd, bucket, 2)` returns 2 bytes (`b = 2`).
  - `bucket` becomes `"\nC\0"`.
  - `reserve = ft_strjoin("AB", "\nC")` joins the strings into `"AB\nC"`. The old `"AB"` reserve is safely freed inside `ft_strjoin`.
- **Iteration 3:**
  - The loop checks `!ft_strchr("AB\nC", '\n')`. It finds the newline and returns false.
  - The loop breaks, `bucket` is freed, and `read_to_reserve` returns `"AB\nC"`.

Step 2: `extract_line` (Line Isolation)

- **Input:** `"AB\nC"`.
- The function iterates through the string until it hits `\n` at index 2.
- It allocates memory `malloc(sizeof(char) * (2 + 2))` (space for characters + newline + null terminator).
- It copies `"AB"` and the `\n` into the new string, returning the final line: `"AB\n"`.

Step 3: `clean_reserve` (Preserving the Leftover)

- **Input:** `"AB\nC"`.
- The function iterates to find the `\n` at index 2.
- It looks at what is left after the newline (the `"C"`).
- It allocates new memory, copies `"C"` into it, and null-terminates it to `"C\0"`.
- Crucially, it calls `free(reserve)` to destroy the old `"AB\nC"` string.
- It returns the new leftover: `"C"`.

Result of Call 1: Returned to user: `"AB\n"` | **Static `reserve` state:** `"C"` (persists to Call 2)

Call 2 — Second Execution

Init: `static char *reserve` enters as `"C"` (carried over from Call 1). `read_to_reserve(fd, "C")` is called.

Step 1: `read_to_reserve` (Hitting EOF)

- **Iteration 1:**
  - `bucket = malloc(3)` is allocated.
  - The loop checks `!ft_strchr("C", '\n')`. It is true (no newline), so it enters.
  - `read(fd, bucket, 2)` attempts to read but hits the End of File (EOF). It returns `0` bytes (`b = 0`).
  - Because `b` is not `> 0`, the `if (b > 0)` block is skipped. `ft_strjoin` is **not** called.
  - The loop condition `b > 0` now fails, so the loop terminates.
  - `bucket` is freed.
  - The function returns the unmodified `reserve`: `"C"`.

Step 2: `extract_line` (Final Line Isolation)

- **Input:** `"C"`.
- The function iterates but hits `\0` before finding a `\n`.
- It allocates memory, copies the `"C"`, and null-terminates it.
- It returns the final string: `"C"`.

Step 3: `clean_reserve` (Resetting the Engine)

- **Input:** `"C"`.
- The function iterates through the reserve. Because there is no `\n`, it reaches the end of the string (`'\0'`).
- The condition `if (!reserve || !reserve[i])` triggers because `reserve[i]` is `'\0'`.
- It calls `free(reserve)` and returns `NULL`.

Result of Call 2: Returned to user: `"C"` | **Static `reserve` state:** `NULL` (fully drained)

Call 3 — Exhaustion

- **Init:** `static char *reserve` enters as `NULL`.
- **Step 1:** `read_to_reserve(fd, NULL)` is called. `read()` returns `0`. The function returns `NULL`.
- **Step 2:** `get_next_line` sees that `reserve` is `NULL` (`if (!reserve)`) and immediately returns `NULL`.

Result of Call 3: Returned to user: `NULL` | **Static `reserve` state:** `NULL`

Execution Summary Table

Call	Reserve In	`read()` Behavior	Line Returned (`line`)	Reserve Out
1	`NULL`	Reads `2` bytes, then `2` bytes	`"AB\n"`	`"C"`
2	`"C"`	Reads `0` bytes (EOF)	`"C"`	`NULL`
3	`NULL`	Reads `0` bytes	`NULL`	`NULL`